

# Sistema TPV — Frutería

Aplicación de consola que simula el funcionamiento de un **TPV (Terminal Punto de Venta)** para una frutería.

El programa permite:

- Consultar un catálogo de frutas.
- Buscar productos por nombre o por coincidencias iniciales.
- Añadir frutas a una cesta.
- Incrementar, reducir o fijar la cantidad en kilogramos.
- Limitar la cantidad máxima de cada producto a 10 kg.
- Cancelar un pedido.
- Generar el ticket de compra.
- Calcular la base imponible y el IVA.
- Realizar el pago en efectivo o con tarjeta.
- Calcular el cambio en pagos en efectivo.
- Generar un ticket final en formato PDF.
- Mantener el catálogo de productos mediante un archivo JSON.

---

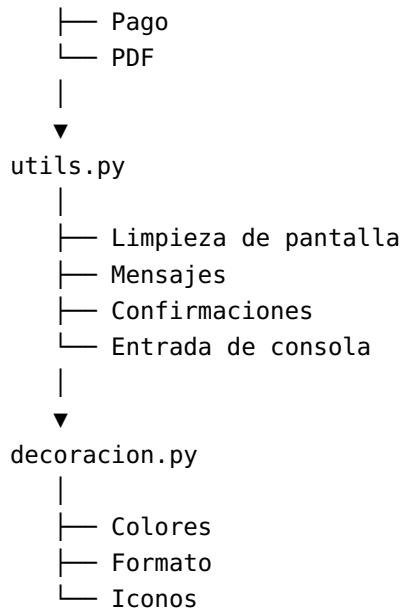
## Estructura del proyecto

```
proyecto/  
|  
├─ Main.py  
├─ fruteria.py  
├─ utils.py  
├─ decoracion.py  
├─ datos.json  
└─ ticket_XXXXXXXX_XXXXXX.pdf
```

Cada archivo tiene una responsabilidad diferente. La aplicación está dividida para evitar concentrar toda la lógica en un único archivo.

La separación principal es:

```
Main.py  
|  
▼  
fruteria.py  
|  
├─ Catálogo y datos  
├─ Lógica de negocio  
├─ Cesta  
└─ Ticket
```



## 1. decoracion.py

Este módulo contiene exclusivamente elementos relacionados con la **presentación visual de la aplicación**.

No contiene lógica de negocio.

### Colores y formato

Se definen constantes ANSI para modificar la apariencia del texto en la terminal:

```
BOLD = "\033[1m"
ROJO = "\033[31;1m"
VERDE = "\033[32m"
NARANJA = "\033[38;5;208m"
AMARILLO = "\033[93m"
CYAN = "\033[96m"
RESET = "\033[0m"
```

Por ejemplo:

```
print(f"{VERDE}Pago aceptado{RESET}")
```

permite mostrar un mensaje en verde y después restaurar el formato normal.

## Control del cursor

También se encuentran las secuencias ANSI:

```
SUBIR = "\033[1A"  
BORRAR = "\033[2K"
```

Estas no son colores.

- `SUBIR` mueve el cursor una línea hacia arriba.
- `BORRAR` elimina el contenido de la línea actual.

Se utilizan para actualizar determinados mensajes de la interfaz sin tener que imprimir constantemente nuevas líneas.

## Banderas

El diccionario:

```
BANDERAS = {  
    "error": "✖",  
    "warning": "⚠",  
    "info": "ℹ",  
    "exito": "✅",  
    "pregunta": "?",  
}
```

centraliza los iconos utilizados para los diferentes tipos de mensajes.

Responsabilidad de `decoracion.py`

```
decoracion.py  
|  
├─ Colores  
├─ Formato de texto  
├─ Control visual del cursor  
└─ Iconos de mensajes
```

---

## 2. `utils.py`

Este módulo contiene funciones **generales de utilidad para la interfaz de consola**.

La idea es que estas funciones puedan utilizarse desde otros módulos sin repetir el mismo código.

## limpiar\_buffer()

Limpia las teclas que hayan quedado pendientes en el buffer de entrada.

La implementación es diferente dependiendo del sistema operativo:

- Windows → `msvcrt`
- Otros sistemas → `select`

Esto permite que la aplicación tenga un comportamiento más independiente del sistema operativo.

---

## limpiar\_pantalla()

Limpia la terminal y vuelve a mostrar la cabecera principal del TPV.

Utiliza:

```
subprocess.run(COMANDO_LIMPIAR, shell=ES_WINDOWS)
```

Después imprime `CABECERA`.

Por tanto, cada vez que se limpia la pantalla, la interfaz vuelve a empezar visualmente desde la cabecera.

---

## mostrar\_mensaje()

Centraliza la presentación de mensajes temporales.

Recibe:

```
mostrar_mensaje(  
    texto,  
    tipo="error",  
    segundos=3,  
    lineas_a_borrar=3  
)
```

La función:

1. Busca el icono correspondiente al tipo de mensaje.
2. Muestra el mensaje.
3. Espera unos segundos.
4. Limpia el buffer de entrada.
5. Borra las líneas utilizadas para mostrar el mensaje.

De esta forma no es necesario repetir este comportamiento en todo el programa.

---

```
pedir_confirmacion()
```

Solicita al usuario una confirmación:

```
? ¿Seguro que quieres cancelar? (s/n):
```

Devuelve:

```
True
```

si el usuario responde afirmativamente y:

```
False
```

en caso contrario.

---

### 3. `fruteria.py`

Este es el **núcleo de la aplicación**.

Aquí se encuentra la lógica relacionada con el catálogo, la cesta, los cálculos, el pago y la generación del ticket.

El módulo no se encarga de controlar el flujo principal de la aplicación. Proporciona funciones que después serán utilizadas por `Main.py`.

---

## 3.1. Configuración y datos

Se definen constantes como:

```
TIPO_IVA = 0.04  
ARCHIVO_DATOS = "datos.json"
```

También existe:

```
CATALOGO_BASE
```

que funciona como catálogo inicial de emergencia.

Cada producto contiene:

```
{  
  "icono": "🍏",  
  "nombre": "Manzana",  
  "precio": 1.95  
}
```

## 3.2. Persistencia del catálogo

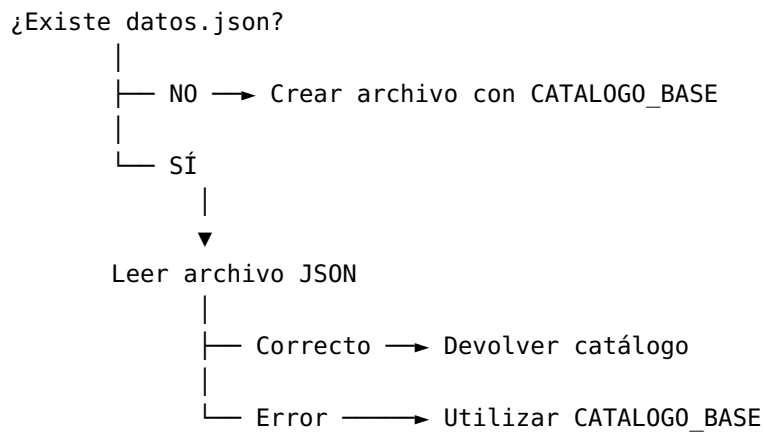
El catálogo se guarda en:

```
datos.json
```

`cargar_catálogo()`

Esta función intenta cargar el catálogo desde el archivo JSON.

El proceso es:



Al importar el módulo se ejecuta:

```
FRUTAS = cargar_catálogo()
```

Por tanto, `FRUTAS` contiene el catálogo que utilizará el programa.

## guardar\_catalogo()

Guarda el catálogo actual en `datos.json`.

Utiliza:

```
json.dump(
    catalogo,
    f,
    ensure_ascii=False,
    indent=4
)
```

`ensure_ascii=False` permite conservar correctamente caracteres como:

á, é, í, ó, ú, ñ

y los emojis.

---

## 3.3. Lógica de la cesta

### mostrar\_cesta()

Muestra el contenido actual de la cesta.

Para cada producto muestra:

 Manzana      2 Kg → 3,90 €

También calcula el total provisional mediante:

```
sum(item["total"] for item in cesta.values())
```

La cesta se representa como un diccionario donde cada clave corresponde a una fruta.

---

### actualizar\_cesta()

Esta función modifica la cesta.

Si el producto ya existe:

```
cesta
├──
│   ├── Manzana
│   │   ├── kg
│   │   └── total
```

se acumulan los nuevos kilos y el nuevo importe.

Si el resultado llega a cero o menos, el producto se elimina de la cesta.

Si el producto todavía no existe y los kilos son positivos, se crea una nueva entrada.

---

## 3.4. Búsqueda de productos

```
buscar_fruta()
```

Permite buscar una fruta introduciendo su nombre o parte inicial.

Primero normaliza la entrada:

```
fruta.lower()
```

y elimina los acentos para facilitar la búsqueda.

Después comprueba qué claves del catálogo empiezan por el texto introducido.

Por ejemplo:

```
man
```

puede encontrar:

```
Manzana
```

El resultado es una lista de coincidencias.

---

## 3.5. Introducción de kilos

```
solicitar_kilos()
```

Esta función controla toda la lógica relacionada con la cantidad de producto.



El usuario puede:

### Añadir

+3,5

Añade 3,5 kg.

### Restar

-2

Reduce 2 kg de la cantidad existente.

### Fijar una cantidad

\*5

Establece directamente la cantidad total en 5 kg.

También controla:

- cantidades negativas inválidas;
- cantidades superiores al límite;
- cantidades con coma decimal;
- cantidades con punto decimal;
- cantidades como `.5`;
- cantidades como `*,5`;
- límite máximo de 10 kg.

---

## 3.6. Comandos globales

`procesar_comando_global()`

Procesa comandos especiales introducidos en lugar del nombre de una fruta.

`/`

Cancela el pedido actual y comienza una nueva cesta después de solicitar confirmación.

`%`

Finaliza la selección de productos y pasa al proceso de ticket y pago.

## ENTER

Muestra información sobre los comandos disponibles.

La función devuelve estados como:

```
CANCELAR  
CONTINUAR  
TICKET  
NINGUNA
```

Esto permite que `Main.py` decida qué hacer sin tener que conocer los detalles internos de cada operación.

---

## 3.7. Formateo de precios

```
formato_precio()
```

Centraliza el formato monetario:

```
4.50
```

se convierte en:

```
4,50 €
```

Esto evita repetir la misma conversión en diferentes partes del programa.

---

## 3.8. Ticket en pantalla

```
mostrar_ticket()
```

Cuando el usuario termina de seleccionar productos, esta función genera una representación del ticket directamente en la terminal.

Calcula:

```
Total  
↓  
Base imponible
```

↓  
IVA

La base imponible se obtiene mediante:

```
base = total / (1 + TIPO_IVA)
```

y el IVA:

```
iva = total - base
```

Después muestra:

- número de ticket;
- productos;
- peso;
- precio por kilogramo;
- total de cada producto;
- base imponible;
- IVA;
- total a pagar.

---

## 3.9. Procesamiento del pago

```
procesar_pago()
```

Permite seleccionar el método de pago.

### Tarjeta

Si el usuario pulsa ENTER:

 PAGO CON TARJETA ACEPTADO

El método devuelto es:

"Tarjeta"

### Efectivo

Si el usuario introduce una cantidad:

```
20
```

se comprueba si es suficiente.

Si:

```
entrega >= total
```

se calcula:

```
cambio = entrega - total
```

Si el dinero es insuficiente, se muestra cuánto falta y se vuelve a solicitar la cantidad.

---

## 3.10. Generación del PDF

```
crear_pdf()
```

Después de realizar el pago se genera un ticket físico en formato PDF.

El archivo tiene el formato:

```
ticket_YYYYMMDD_HHMMSS.pdf
```

Por ejemplo:

```
ticket_20260909_141530.pdf
```

Para generar el documento se utiliza la biblioteca `reportlab`.

El PDF contiene:

- nombre de la frutería;
- CIF;
- dirección;
- fecha y hora;
- número de ticket;
- productos;
- peso;
- PVP;
- total;
- base imponible;

- IVA;
  - total a pagar;
  - método de pago;
  - importe entregado;
  - cambio, cuando corresponde.
- 

## 4. Main.py

`Main.py` es el **punto de entrada y controlador principal de la aplicación**.

Su función principal es coordinar los diferentes módulos.

No debería contener la lógica detallada de:

- cálculo de precios;
- gestión de la cesta;
- búsqueda de productos;
- cálculo del IVA;
- generación del PDF.

Estas responsabilidades pertenecen a `fruteria.py`.

`Main.py` se encarga principalmente del **flujo de ejecución**.

---

### 4.1. Inicio del programa

Cuando se ejecuta:

```
python Main.py
```

Python llega a:

```
if __name__ == "__main__":  
    ejecutar_tpv()
```

y comienza la ejecución de la aplicación.

---

### 4.2. Creación de una nueva cesta

Al iniciar un pedido:

```
cesta = {}
```

se crea una cesta vacía.

La cesta pertenece al pedido actual.

Cuando el pedido termina, se puede crear una nueva cesta para el siguiente cliente.

---

## 4.3. Selección de fruta

Se limpia la pantalla:

```
limpiar_pantalla()
```

y se muestra la cesta actual.

Después se solicita:

```
¿Qué fruta quieres?:
```

La entrada pasa a:

```
core.procesar_comando_global()
```

para comprobar primero si se trata de un comando especial.

Si no es un comando, se utiliza:

```
core.buscar_fruta()
```

para localizar el producto.

---

## 4.4. Coincidencias

Existen tres posibilidades.

### Una coincidencia

Se obtiene el producto:

```
clave = coincidencias[0]
```

y se recuperan:

```
icono  
nombre  
precio
```

Después se solicita la cantidad.

### **Varias coincidencias**

Se informa al usuario de que debe especificar mejor el nombre.

### **Ninguna coincidencia**

Se muestra un mensaje indicando que la fruta no existe.

---

## **4.5. Introducción y actualización del peso**

Una vez identificada la fruta:

```
kg = core.solicitar_kilos(...)
```

se obtiene la cantidad que debe añadirse o restarse.

Después:

```
core.actualizar_cesta(...)
```

actualiza la cesta.

Este ciclo se repite hasta que el usuario introduce:

```
%
```

para finalizar el pedido.

---

## **4.6. Finalización del pedido**

Cuando:

```
accion == "TICKET"
```

se abandona el ciclo de selección de productos.

Se genera un identificador:

```
id_ticket = datetime.now().strftime("%Y%m%d_%H%M%S")
```

Después se calcula el total y se muestra el ticket:

```
core.mostrar_ticket(cesta, id_ticket)
```

---

## 4.7. Pago

A continuación:

```
core.procesar_pago(total_a_pagar)
```

gestiona el pago.

La función devuelve:

```
metodo_pago  
entrega  
cambio
```

Estos datos serán necesarios para generar el ticket PDF.

---

## 4.8. Generación del PDF

Después del pago:

```
core.crear_pdf(  
    cesta,  
    id_ticket,  
    entrega,  
    cambio,
```



```
        metodo_pago  
    )
```

genera el documento PDF.

Finalmente se informa al usuario de que el archivo ha sido creado correctamente.

## 4.9. Nuevo pedido o salida

Después de completar una venta se muestra:

```
[/] Nuevo pedido | [Ctrl + C] Salir
```

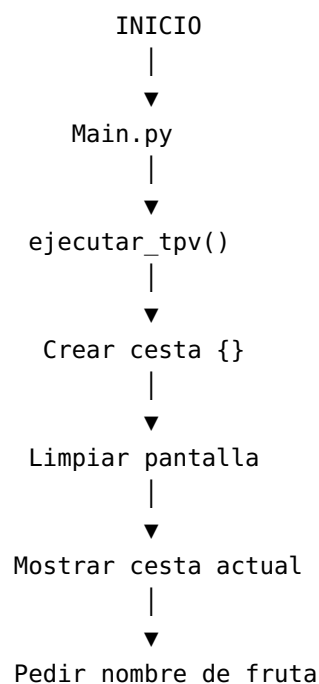
Si el usuario introduce `/`, comienza un nuevo pedido.

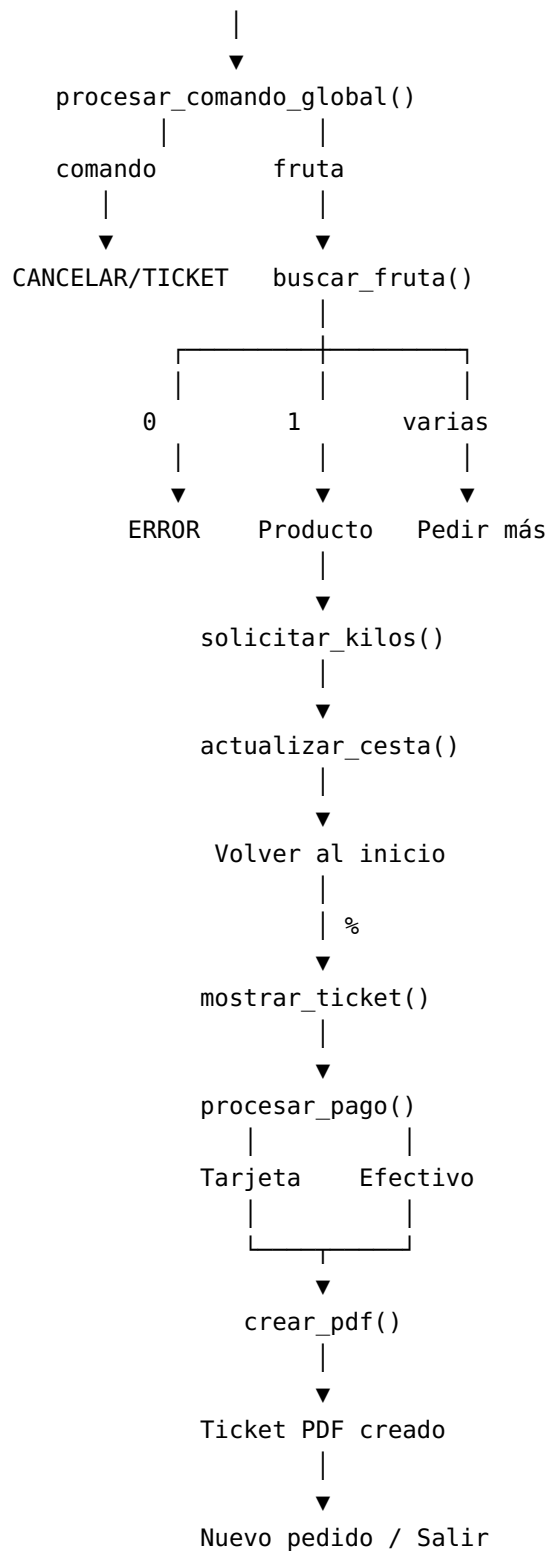
Si pulsa `Ctrl + C`, se produce una interrupción `KeyboardInterrupt` y el programa termina mostrando:

```
¡Hasta pronto! 🙌
```

## Flujo completo de la aplicación

El funcionamiento general puede representarse así:

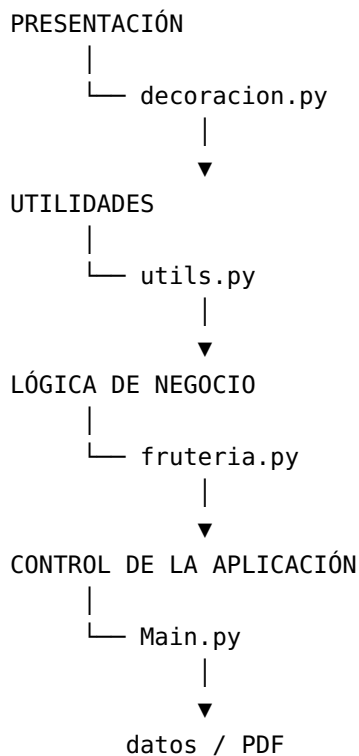




## Responsabilidad de cada módulo

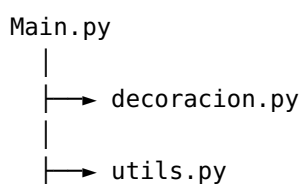
| Archivo                    | Responsabilidad                              |
|----------------------------|----------------------------------------------|
| <code>Main.py</code>       | Controlar el flujo general de la aplicación  |
| <code>fruteria.py</code>   | Contener la lógica de negocio                |
| <code>utils.py</code>      | Proporcionar funciones auxiliares de consola |
| <code>decoracion.py</code> | Centralizar colores, formato e iconos        |
| <code>datos.json</code>    | Almacenar el catálogo de frutas              |

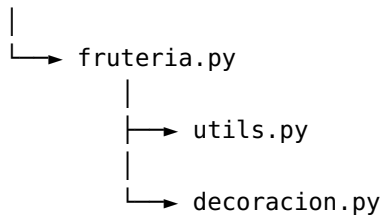
La idea principal es aplicar una separación de responsabilidades:



## Dependencias entre módulos

Las dependencias principales son:





`fruteria.py` funciona como el módulo central de la lógica, mientras que `Main.py` actúa como controlador.

De esta manera, `Main.py` no necesita conocer cómo se calcula un precio, cómo se busca una fruta o cómo se genera un PDF. Solo necesita llamar a las funciones correspondientes.

## Objetivo de la división

La división del programa en módulos permite:

- reducir el tamaño de cada archivo;
- facilitar la lectura del código;
- localizar más rápidamente los errores;
- reutilizar funciones;
- separar interfaz y lógica de negocio;
- facilitar futuras modificaciones;
- evitar duplicación de código;
- hacer que cada módulo tenga una responsabilidad clara.

Por ejemplo, si se quiere modificar el sistema de colores, se puede trabajar en:

`decoracion.py`

sin modificar la lógica de la cesta.

Si se quiere cambiar el formato del PDF, se puede modificar:

`fruteria.py` → `crear_pdf()`

sin cambiar el flujo principal de `Main.py`.

Si se quiere cambiar la forma de limpiar la consola, se puede modificar:

`utils.py` → `limpiar_pantalla()`

sin tocar la lógica de los productos.

## Tecnologías utilizadas

- **Python 3**
- **JSON** para persistencia del catálogo
- **ANSI Escape Sequences** para la interfaz de terminal
- **ReportLab** para la generación de tickets PDF
- `datetime` para generar identificadores y fechas
- `os` para comprobar la existencia del archivo JSON
- `subprocess` para ejecutar comandos de la terminal
- `msvcrt` / `select` para gestionar la entrada dependiendo del sistema operativo

## Ejecución

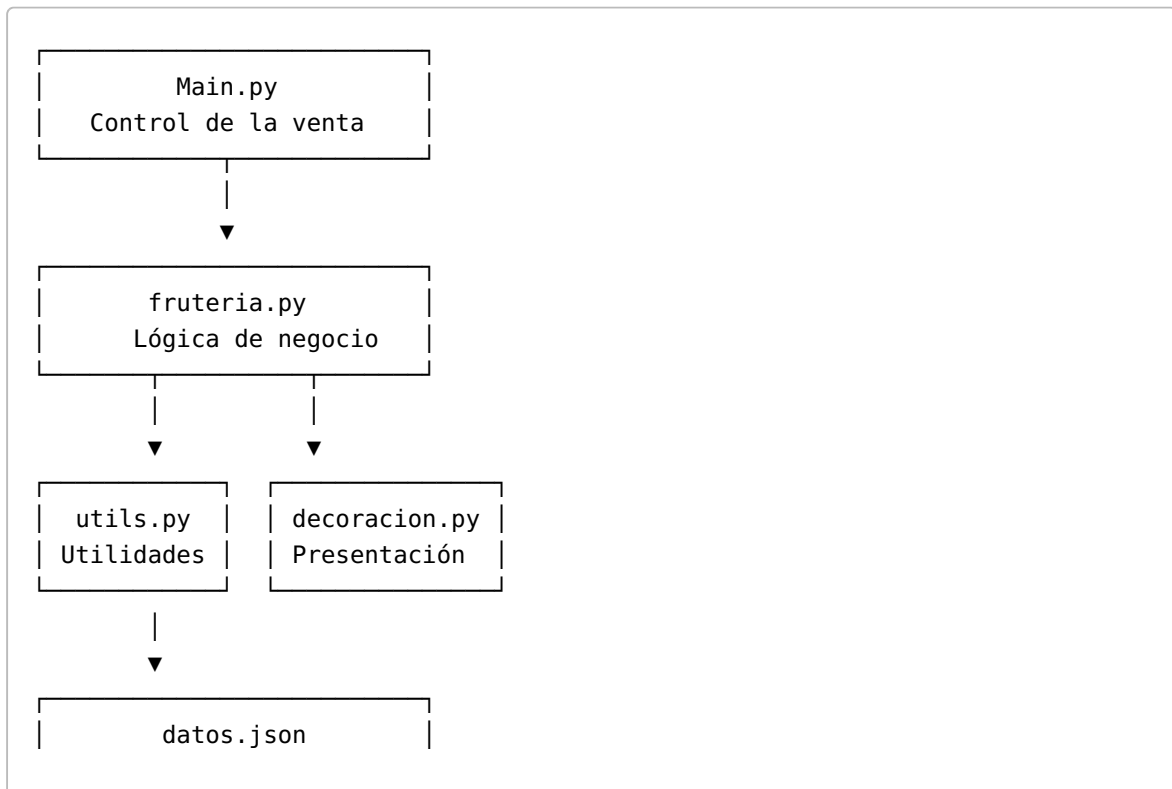
Desde la carpeta del proyecto:

```
python Main.py
```

El programa iniciará el TPV y mostrará el catálogo disponible.

## Resumen de la arquitectura

El proyecto sigue una estructura sencilla basada en la **separación de responsabilidades**:



Catálogo persistente

La aplicación queda así dividida en cuatro niveles conceptuales:

1. **Control** → `Main.py`
2. **Lógica de negocio** → `fruteria.py`
3. **Utilidades** → `utils.py`
4. **Presentación** → `decoracion.py`

Esto permite mantener el código organizado y facilita la ampliación del proyecto en el futuro.